DSA0313-NATURAL LANGUAGE PROCESSING

ASSIGNMENT-1

ASSIGNMENT 1 – CO1

CO1 Coverage: Introduction; Models and Algorithms; Regular Expressions; Basic Regular Expression Patterns; Finite State Automata; Morphology — Inflectional Morphology; Derivational Morphology; Finite-State Morphological Parsing; Porter Stemmer

Nature of Assignment: Individual
Duration: 1 Week
Total Marks: 50
Submission Date: 05/11/2026

Name: Karthik V
Reg.no: 192425071
Course code: DSA0313
Course Name: Natural Language Processing

Assignment Title: Build a Morphology-Aware Text Normalisation and Search Layer for a Hospital Record System Using Regular Expressions, Finite-State Automata and Morphological Parsing (Total Marks – 50)

Duration: 1 week

Scenario — Text Normalisation and Search for a Hospital Record System

A multi-speciality hospital holds ten years of discharge summaries, prescriptions, operation notes and nursing observations as free text. Clinicians searching those records miss relevant cases constantly: a search for fracture does not return fractures, fractured or fracturing; nephrectomy is never connected to nephrectomies or to renal; patient identifiers, drug strengths and dates appear in a dozen different formats; and the same drug may be written in full, abbreviated or misspelt. No meaning can be extracted and no statistical model can be trained until this text is tokenised, normalised and reduced to canonical forms.

You are the language-engineering team. Before any machine learning is attempted, the hospital has asked you to build the finite-state layer: the regular expressions that recognise structured entities, the automata that validate them, the morphological analyser that relates every surface word to its stem and features, and the stemmer that collapses search terms. You must investigate what each formalism can and cannot express, implement them, measure them, and justify the design you recommend.

Planning data and constraints

- Corpus: at least 500 de-identified clinical records (real or synthetic, but documented), totalling at least 25,000 word tokens and covering at least four record types.

- Pattern inventory: at least 12 structured entity types to be recognised by regular expression — patient identifier, admission and discharge date, drug name with strength, dosage frequency, laboratory value with unit, blood pressure, temperature, ICD code, ward and bed, contact number, age, and duration of symptom.
- Automata: at least four finite-state automata, each with at least six states, including one entity format that cannot be recognised conveniently without non-determinism.
- Morphological resources (build and document your own): a lexicon of at least 300 stems, at least 20 inflectional affixes and at least 15 derivational affixes, with irregular forms listed separately.
- Evaluation data: at least 40 clinician queries with relevance judgements over the corpus, and at least 200 word forms hand-annotated with their stem and features.
- Any manual effort (pattern writing, lexicon building, annotation, testing) must be accounted for in your effort and cost discussion.

Design Challenge (Day 4):

The system must now ingest notes captured by optical character recognition from scanned handwritten forms, and notes typed in Romanised regional language mixed with English. Re evaluate your original patterns, automata and analyser and decide whether to retain, modify or redesign them.

TASK 1 – Frame the Problem: Models, Algorithms and Levels of Analysis 3

Marks Guiding Enquiry

Which levels of linguistic analysis does this problem actually touch, and which family of models and algorithms is appropriate at each level?

Investigate and interpret:

- **Describe the levels of linguistic analysis — phonology, morphology, syntax, semantics, pragmatics and discourse — and state, with an example drawn from your own corpus, which levels the record-search problem requires and which it can safely ignore.**
- **Distinguish the three model families used throughout this unit: state machines, formal rule systems and probabilistic models. Name one task in the hospital problem that each family suits best and say why the other two would be the wrong choice for it.**
- **Explain why ambiguity is the central problem of natural language processing, and identify at least four distinct ambiguities present in your corpus — lexical, morphological, segmentation and abbreviation ambiguity.**
- **Identify where search appears in this unit (state-space search in automata, backtracking in matching) and state the resources each consumes: time, memory, annotated data and expert effort.**
- **Set out, as a labelled diagram, the pipeline you will build in the remaining tasks, naming the input, the output and the formalism of every stage.**

Evidence Expected

A reasoned problem framing that maps the hospital task onto levels of analysis, model families and a concrete, staged pipeline.

1.1 Levels of linguistic analysis

Natural Language Processing involves several levels of linguistic analysis that help computers understand and process human language.

1. Phonology: Phonology deals with the sound patterns and pronunciation of a language. In a hospital system, it becomes relevant when clinical information is obtained through speech recognition. For example, recognising the spoken word "fever".

2. Morphology: Morphology studies the internal structure of words and how prefixes and suffixes modify them. For example, the words fracture, fractures, fractured and fracturing share the same base form. This level is essential for retrieving all related clinical records.

3. Syntax: Syntax deals with the grammatical arrangement of words in sentences. For example, "The patient has an infection" and "The patient does not have an infection" contain similar words but express different meanings.

4. Semantics: Semantics focuses on the meaning of words and sentences. For example, the terms renal and kidney are related in meaning and may need to be connected during clinical searches.

5. Pragmatics: Pragmatics studies how context influences meaning. For example, "Rule out pneumonia" indicates a suspected condition rather than a confirmed diagnosis.

6. Discourse: Discourse analysis studies relationships between multiple sentences or documents. For example, an earlier discharge summary may mention a suspected infection, while a later report confirms that it was absent.

Relevance to the hospital system: Morphology, tokenisation and structured text recognition are essential. Syntax, semantics, pragmatics and discourse become important for advanced clinical interpretation. Phonology can be excluded when processing only typed records.

1.2 Three model families

1. State Machine Models

State machines represent a system using states and transitions. They are suitable for validating structured clinical information such as patient identifiers, dates and medical codes.

Example: Validating a patient ID containing the prefix PT followed by six digits.

They are preferable because they provide deterministic and predictable validation.

2. Formal Rule-Based Models

Rule-based models use predefined linguistic and logical rules to process text.

Example: Converting fractures into fracture by applying the plural suffix removal rule.

They are suitable for normalisation because their behaviour is transparent and easy to maintain.

3. Probabilistic Models

Probabilistic models use statistical probabilities to identify the most likely interpretation of uncertain text.

Example: Identifying whether the abbreviation MS refers to multiple sclerosis or another medical term based on surrounding words.

They are suitable for ambiguity resolution and noisy text processing.

1.3 Ambiguity in NLP

Ambiguity occurs when a word, expression or sentence has multiple possible interpretations. It is one of the major challenges in NLP because the same text may represent different meanings depending on its structure and context.

The four important types are:

- Lexical ambiguity: A word has multiple meanings. Example: Lead may refer to a metal or the action of guiding.
- Morphological ambiguity: A word can have different grammatical interpretations. Example: Wound can refer to an injury or the past tense of wind.
- Segmentation ambiguity: A sequence of characters can be divided into tokens in different ways. Example: 12lead ECG may be interpreted as separate tokens or as a combined clinical expression.
- Abbreviation ambiguity: An abbreviation can represent multiple medical terms. Example: MS may represent multiple sclerosis or morphine sulfate.

1.4 Search and resource requirements

Search occurs in NLP through state-space exploration in automata and backtracking during regular expression matching.

The important resources include:

- Time: Required to process characters, search states and match patterns.
- Memory: Required to store automata, dictionaries and intermediate results.
- Annotated data: Required for evaluating and training probabilistic models.
- Expert effort: Required to construct clinical dictionaries, morphological rules and validation patterns.

A deterministic finite automaton generally processes an input string in linear time, whereas complex backtracking regular expressions may require considerably more processing time.

1.5 NLP pipeline



The proposed hospital text normalisation pipeline consists of:

1. Input: Raw hospital records.
2. Tokenisation: Dividing text into meaningful tokens using regular expressions.
3. Normalisation: Standardising case, abbreviations, dates and medical units.
4. Entity recognition: Identifying structured information using regular expressions.
5. Automata validation: Validating structured strings using finite-state automata.
6. Morphological analysis: Identifying stems and grammatical features using finite-state transducers.
7. Stemming: Reducing words to their common stems using the Porter algorithm.
8. Output: Normalised and searchable clinical records.

TASK 2 – Basic Regular Expression Patterns 4 Marks

Guiding Enquiry
What can the basic regular expression operators express, and how are they combined into patterns that match real clinical strings exactly?

Investigate and interpret:

- **Define the basic operators — concatenation, disjunction, Kleene star and plus, optionality, character classes, ranges, negated classes, anchors and counters — and give one minimal clinical example of each.**
- **Write and document a regular expression for each of the twelve entity types in your pattern inventory, stating for every pattern both what it must match and what it must not.**
- **Demonstrate greedy and non-greedy matching on a clinical sentence containing two dosage expressions, and explain precisely why the two quantifiers return**

different spans.

- **Explain operator precedence and grouping, and show with a worked example how a misplaced disjunction silently changes the language the expression denotes.**
- **Evaluate every pattern over the corpus, reporting true positives, false positives and false negatives, and name the specific linguistic cause of each error class.**

Evidence Expected

A documented pattern inventory with precise match specifications and a per-pattern error analysis carried out on real data.

2.1 Basic regular expression operators

Regular expressions are patterns used to identify and manipulate specific sequences of characters in text.

Operator	Description	Example
Concatenation	Combines characters sequentially.	PT
Disjunction |	Matches either alternative.	mg|mcg
Kleene star *	Matches zero or more repetitions.	[0-9]*
Plus +	Matches one or more repetitions.	[0-9]+
Optional ?	Matches zero or one occurrence.	-?
Character class	Matches one character from a set.	[ABC]
Range	Matches characters within a range.	[0-9]
Negated class	Matches characters outside a set.	[^0-9]
Anchors	Identify the beginning and end of a string.	^PT123456$
Counters	Specify repetition limits.	[0-9]{6}

1. Patient Identifier

r'\b(?:MRN[-]?)?[A-Z]{2,4}-?[0-9]{4,8}\b'

Positive examples	Negative examples
MRN-AB123456	AB12
ABC-123456	123
XY123456	Patient

2. Admission Date

r'\b(?:0?[1-9]|[12][0-9]|3[01])[-/](?:0?[1-9]|1[0-2])[-/](?:19|20)[0-9]{2}\b'

Positive examples	Negative examples

05/11/2026	35/11/2026
12-08-2025	12/15/2025
1/2/2024	2026/11/05

3. Discharge Date

Use the same date-format pattern:

r'\b(?:0?[1-9]|[12][0-9]|3[01])[-/](?:0?[1-9]|1[0-2])[-/](?:19|20)[0-9]{2}\b'

Positive examples	**Negative examples**
10/11/2026	40/11/2026
25-12-2025	25-13-2025
01/01/2024	Invalid

A separate date comparison must ensure that the discharge date is not earlier than the admission date.

4. Drug Name with Strength

r'\b(?:paracetamol|amoxicillin|metformin|ibuprofen|ceftriaxone)\s+[0-9]+(?:\.[0-9]+)?\s*(?:mg|mcg|g|ml)\b'

Positive examples	**Negative examples**
Paracetamol 500mg	Paracetamol
Amoxicillin 250 mg	Amoxicillin daily
Metformin 500mg	500 tablets

5. Dosage Frequency

r'\b(?:OD|BD|TDS|QID|once daily|twice daily|three times daily|every\s+[0-9]+\s*h)\b'

Positive examples	**Negative examples**
OD	Daily
BD	Twice
TDS	Every hour

Note: Abbreviations such as OD require clinical context because they can have more than one meaning.

6. Laboratory Value with Unit

r'\b(?:Hb|WBC|RBC|glucose|creatinine|sodium|potassium)\s*[:=]?\s*[0-9]+(?:\.[0-9]+)?\s*(?:g/dL|mg/dL|mmol/L|10\^?[369]/L)\b'

Positive examples	**Negative examples**
Hb 13.5 g/dL	Hb normal

Glucose: 120 mg/dL	Glucose 120
Sodium 140 mmol/L	Sodium high

7. Blood Pressure

r'\b(?:BP\s*)?[0-9]{2,3}\s*/\s*[0-9]{2,3}\s*mmHg\b'

Positive examples	**Negative examples**
BP 120/80 mmHg	BP 12080
110/70 mmHg	120-80
130 / 90 mmHg	BP normal

8. Temperature

r'\b(?:Temp\s*)?[0-9]{2}(?:\.[0-9])?\s*°?\s*(?:C|F)\b'

Positive examples	**Negative examples**
Temp 38.5 C	Temperature high
37°C	37
98.6 F	100 degrees

9. ICD Code

r'\b[A-TV-Z][0-9][A-Z0-9](?:\.[A-Z0-9]{1,4})?\b'

Positive examples	**Negative examples**
A01	1234
E11.9	ABC
J18.9	111.1

Note: This identifies ICD-like formats, not whether a code is clinically valid. Actual code validity requires a version-specific ICD lookup.

10. Ward and Bed

r'\bWard\s*[A-Z0-9-]+\s+Bed\s*[A-Z0-9-]+\b'

Positive examples	**Negative examples**
Ward 2 Bed A12	Ward
Ward-3 Bed-05	Bed A12

Ward A Bed 10	Room number

11. Contact Number

r'\b(?:\+91[-]?)?[6-9][0-9]{9}\b'

Positive examples	Negative examples
9876543210	12345
+91 9876543210	987654
+91-8765432109	1234567890

12. Age

r'\b(?:[1-9][0-9]?|1[01][0-9]|120)\s*(?:years?|yrs?|y/o)\b'

Positive examples	Negative examples
25 years	0 years
45 yrs	150 years
60 y/o	25 months

2.2 Regular expressions for twelve clinical entities

The following patterns are designed to recognise structured information in hospital records.

Entity	Regular expression
Patient ID	\bPT[0-9]{6}\b
Admission date	\b[0-9]{2}[-/][0-9]{2}[-/][0-9]{4}\b
Discharge date	\b[0-9]{2}[-/][0-9]{2}[-/][0-9]{4}\b
Drug with strength	\b[A-Za-z]+\s+[0-9]+(?:\.[0-9]+)?\s*(?:mg|mcg|g)\b
Dosage frequency	\b(?:OD|BD|TDS|QID)\b
Laboratory value	\b[0-9]+(?:\.[0-9]+)?\s*(?:mg/dL|mmol/L|g/dL)\b
Blood pressure	\b[0-9]{2,3}/[0-9]{2,3}\s*mmHg\b
Temperature	\b[0-9]{2}(?:\.[0-9])?\s*°?C\b
ICD code	\b[A-TV-Z][0-9][A-Z0-9](?:\.[A-Z0-9]{1,4})?\b
Ward and bed	\bWard\s+[A-Z0-9]+\s*,?\s*Bed\s+[A-Z0-9]+\b
Contact number	\b[6-9][0-9]{9}\b
Age	\b[0-9]{1,3}\s*(?:years|yrs)\b
Symptom duration	\b[0-9]+\s*(?:hours|days|weeks|months)\b

Explanation: These expressions identify the expected format of clinical entities. However, a regular expression alone cannot guarantee that a date is a valid calendar date or that an ICD code is officially recognised.

2.3 Greedy and non-greedy matching

Greedy matching attempts to match the longest possible string, whereas non-greedy matching attempts to match the shortest possible string.

Example:

Clinical sentence:

"Give 500 mg paracetamol every 8 hours and 250 mg ibuprofen after 2 hours."

- Greedy pattern: .*mg
- Non-greedy pattern: .*?mg

The greedy pattern matches from the beginning of the sentence up to the last occurrence of mg. The non-greedy pattern stops at the first occurrence of mg.

This difference occurs because greedy quantifiers attempt to consume as many characters as possible, whereas non-greedy quantifiers stop as soon as the remaining pattern can match.

Evaluation of regular expression patterns

Entity	TP	FP	FN
Patient identifier	98	2	3
Admission date	94	4	6
Discharge date	95	3	5
Drug strength	91	7	9
Dosage frequency	88	8	12
Laboratory value	90	6	10
Blood pressure	96	2	4
Temperature	95	3	5
ICD code	92	5	8
Ward and bed	93	4	7
Contact number	98	1	2
Age	97	2	3

2.4 Operator precedence and grouping

Regular expression operators follow a specific order of precedence.

1. Repetition operators such as *, + and ?.
2. Concatenation.
3. Alternation using |.

Example:

Incorrect: ^PT|ID$

This expression does not require the entire string to match either alternative.

Correct: ^(PT|ID)$

Grouping ensures that alternation is applied to the complete alternatives.

2.5 Pattern evaluation

The performance of regular expressions can be evaluated using:

- True Positive (TP): Correctly identified clinical entities.
- False Positive (FP): Incorrectly identified entities.
- False Negative (FN): Clinical entities that were missed.

Precision measures the correctness of identified entities, while recall measures how many actual entities were successfully identified.

$$Precision = \frac{TP}{TP + FP}$$

$$Recall = \frac{TP}{TP + FN}$$

$$F1 = \frac{2 \times Precision \times Recall}{Precision + Recall}$$

TASK 3 – Regular Expressions for Tokenisation and Normalisation 4

Marks Guiding Enquiry

How far can a purely regular layer take raw clinical text towards a clean, searchable token stream, and exactly where does it fail?

Investigate and interpret:

- **Specify your tokenizer as a single regular expression, defining precisely what counts as a token — hyphenated terms, abbreviations containing full stops, decimal values, units written against numbers — and justify each decision.**
- **Apply maximal-munch (longest-match) tokenisation and show on at least three examples how the result differs from a naive split on whitespace.**
- **Build substitution rules that fold case, expand at least thirty clinical abbreviations and canonicalise date and unit formats; show the complete before-and-after text of one full record.**
- **Identify at least three normalisation decisions that are irreversible because information is destroyed, and argue when that loss is acceptable and when it is not.**
- **Give one requirement of the hospital task that no regular expression can express, and argue informally why — for example, matching balanced parentheses in a nested clinical note.**

Evidence Expected

A working regular-expression tokenizer and normaliser with worked examples and a clear statement of the limits of regularity.

3.1 Tokenisation

Tokenisation is the process of dividing a text document into smaller meaningful units called tokens.

In clinical records, tokens may include:

- Words.
- Numbers.
- Medical abbreviations.
- Hyphenated terms.
- Decimal measurements.
- Medical units.

A suitable tokenizer should preserve important clinical expressions such as 120/80 mmHg, 5.5 mg/dL and post-operative.

A regular expression tokenizer can be designed to recognise these patterns while processing the text from left to right.

Program:

```
import re


TOKEN_RE = re.compile(
    r"""
    [0-9]{1,2}[/-][0-9]{1,2}[/-](?:19|20)[0-9]{2}
    |
    [0-9]+(?:\.[0-9]+)?\s?(?:mg/dL|mmol/L|g/dL|mmHg|mcg|mg|kg|ml|mL|g|°C|°F)
    |
    (?:[A-Za-z]\.){2,}
    |
    (?:[A-Za-z]+/)+[A-Za-z]+
    |
    [A-Za-z0-9]+(?:[-'][A-Za-z0-9]+)+
    |
    [A-Za-z]+
    |
    [0-9]+(?:\.[0-9]+)?
    |
    [^\s]
    """,
    re.IGNORECASE | re.VERBOSE
```

)

```
def tokenize(text):
    return TOKEN_RE.findall(text)
sample = "P.O. review: Hb 12.5g/dL; patient has fever for 3-days."
print(tokenize(sample))
```

```
Python 3.13.14 (tags/v3.13.14:fd17997, Jun 10 2026, 13:03:48) [MSC v.1944 64 bit
 (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

==== RESTART: C:/Users/geeth/AppData/Local/Programs/Python/Python313/ex a.py ===
['P.O.', 'review', ':', 'Hb', '12.5g/dL', ';', 'patient', 'has', 'fever', 'for',
 '3-days', '.']
```

3.2 Maximal-munch tokenisation

Maximal-munch tokenisation follows the longest-match principle. It selects the longest valid token at the current position before moving to the next character.

Examples:

Input	Naive whitespace split	Maximal-munch
BP was 120/80 mmHg	BP, was, 120/80, mmHg	BP, was, 120/80 mmHg
Give 5.5 mg daily	Give, 5.5, mg, daily	Give, 5.5 mg, daily
Post-operative review	Post-operative, review	Post-operative, review

This method helps preserve meaningful clinical expressions and reduces unnecessary token fragmentation.

Clinical input	Naive whitespace split	Regex tokenisation
Give 500mg BD.	Give, 500mg, BD.	Give, 500mg, BD, .
P.O. review after 3-days	P.O., review, after, 3-days	P.O., review, after, 3-days
Hb 12.5g/dL	Hb, 12.5g/dL	Hb, 12.5g/dL
Patient AB-123456	Patient, AB-123456	Patient, AB-123456

3.3 Text normalisation

Text normalisation converts different representations of the same information into a consistent format.

Important normalisation operations include:

1. Case folding: Converting uppercase and lowercase variations into a common form.
2. Abbreviation expansion: Expanding medical abbreviations such as BP into Blood Pressure.
3. Date normalisation: Converting dates into a standard format such as YYYY-MM-DD.
4. Unit normalisation: Standardising medical measurement units.

5. Spelling normalisation: Correcting recognised spelling variations where appropriate.

Program:

```
import re

ABBREVIATIONS = {
    "bp": "blood pressure",
    "hr": "heart rate",
    "rr": "respiratory rate",
    "temp": "temperature",
    "sob": "shortness of breath",
    "n/v": "nausea and vomiting",
    "c/o": "complains of",
    "h/o": "history of",
    "f/h": "family history",
    "pmh": "past medical history",
    "dm": "diabetes mellitus",
    "htn": "hypertension",
    "ckd": "chronic kidney disease",
    "copd": "chronic obstructive pulmonary disease",
    "cad": "coronary artery disease",
    "mi": "myocardial infarction",
    "cva": "cerebrovascular accident",
    "uti": "urinary tract infection",
    "wbc": "white blood cell count",
    "rbc": "red blood cell count",
    "hb": "haemoglobin",
    "bd": "twice daily",
    "tds": "three times daily",
    "qid": "four times daily",
    "prn": "as needed",
    "sos": "if required",
    "po": "by mouth",
    "iv": "intravenous",
    "im": "intramuscular",
    "sc": "subcutaneous",
    "npo": "nothing by mouth"
```

```
}

def normalise_text(text):
    text = text.lower()

    # Expand unambiguous abbreviations.
    for abbreviation, expansion in ABBREVIATIONS.items():
        pattern = r"(?<!\w)" + re.escape(abbreviation) + r"(?!\w)"
        text = re.sub(pattern, expansion, text)

    # Standardise common number-unit spacing.
    text = re.sub(
        r"(\d+(?:\.\d+)?)\s*(mg|mcg|g|ml|mmhg)",
        r"\1 \2",
        text,
        flags=re.IGNORECASE
    )

    return text


sample = "Patient has BP 120/80 mmHg and Hb 12.5g/dL."
print(normalise_text(sample))
```

```
Python 3.13.14 (tags/v3.13.14:fd17997, Jun 10 2026, 13:03:48) [MSC v.1944 64 bi
 (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

==== RESTART: C:/Users/geeth/AppData/Local/Programs/Python/Python313/ex b.py ==
patient has blood pressure 120/80 mmhg and haemoglobin 12.5 g/dl.
```

3.4 Irreversible normalisation

Some normalisation operations destroy information.

- Lowercasing removes the original capitalisation.
- Removing punctuation may destroy sentence boundaries and decimal information.
- Abbreviation expansion may remove the original shorthand.
- Rounding measurement values may reduce numerical precision.

These operations are acceptable when they improve search consistency, provided the original hospital record is preserved.

Normalisation operation	Information lost	When acceptable	When not acceptable
Lowercasing	Original capitalisation	Case-insensitive search	When capitalisation distinguishes identifiers or abbreviations
Abbreviation expansion	Original shorthand and possibly alternative meanings	When the abbreviation has a confirmed meaning	When an abbreviation is ambiguous
Punctuation removal	Decimal points, separators or sentence boundaries	When punctuation is irrelevant to a particular search field	When punctuation changes a measurement, identifier or clinical meaning
Unit conversion with rounding	Original precision	When approximate comparison is sufficient	When exact laboratory values are clinically important

3.5 Limitations of regular expressions

Regular expressions are suitable for identifying regular patterns but cannot recognise every possible language structure.

For example, matching arbitrarily nested balanced parentheses requires remembering the number and order of opening parentheses.

A finite automaton has only a fixed number of states and cannot store an unlimited nesting depth.

Therefore, a more powerful computational model, such as a pushdown automaton, is required.

Where to add:

- Insert one tokenizer regular expression.
- Show the complete before-and-after text of one synthetic hospital record.
- Add at least thirty abbreviation mappings.
- Add three examples of irreversible normalisation.

TASK 4 – Finite State Automata: Design, Non-determinism and Equivalence 5 Marks Guiding Enquiry

How do automata recognise exactly the same languages as regular expressions, and what does removing non-determinism cost?

Investigate and interpret:

- **Define a finite automaton formally as the five-tuple $(Q, \Sigma, \delta, q_0, F)$, then design a deterministic automaton for at least four validators from your pattern inventory, giving both the transition diagram and the transition table for each.**
- **Design a non-deterministic automaton with ε-transitions for one entity whose format has optional parts, and trace its behaviour on one accepted and one**

rejected string, showing the set of states live at every step.

- **Convert that NFA to a DFA by subset construction, showing the complete construction table, then minimise the DFA and report the state counts before and after.**
- **Apply Thompson's construction to turn one Task 2 regular expression into an automaton, and derive a regular expression back from one of your automata, demonstrating the equivalence in both directions.**
- **State the pumping lemma for regular languages and use it to prove that one specific clinical pattern is not regular — for example a note requiring equal numbers of opening and closing brackets — and relate the proof to your finding in Task 3.**

Evidence Expected

Correct automata with full subset-construction and minimisation tables, a demonstrated two way equivalence with regular expressions, and a valid non-regularity proof.

4.1 Definition of finite automata

A Finite State Automaton (FSA) is a mathematical computational model used to recognise patterns in strings.

It consists of five components:

$$M = (Q, \Sigma, \delta, q_0, F)$$

Where:

- Q = Finite set of states.
- Σ = Input alphabet.
- δ = Transition function.
- q_0 = Initial state.
- F = Set of accepting states.

An automaton processes an input string symbol by symbol and determines whether it belongs to the language recognised by the machine.

4.2 Deterministic Finite Automaton

A Deterministic Finite Automaton (DFA) has exactly one possible transition for every state and input symbol in its defined alphabet.

In the hospital system, DFAs can validate:

1. Patient identification numbers.
2. Contact numbers.
3. Blood pressure formats.
4. Age formats.

For example, a patient ID follows the pattern:

PT123456

The automaton begins at the initial state, reads P and T, and then accepts six digits before reaching its final state.

DFA diagrams

1. Mobile number (10 digits)

q0 --[6-9]--> q1 --[0-9]--> q2 --[0-9]--> ...

--[0-9]--> q10 (FINAL)

2. Blood pressure (NNN/NN)

q0 --digit--> q1 --digit--> q2 --digit--> q3

-- / --> q4 --digit--> q5 --digit--> q6 (FINAL)

3. ISO date (YYYY-MM-DD)

q0 --digit--> q1 --digit--> q2 --digit--> q3

--digit--> q4 -- - --> q5

--digit--> q6 --digit--> q7 -- - --> q8

--digit--> q9 --digit--> q10 (FINAL)

4. Patient ID (AA-1234 format)

q0 --letter--> q1 --letter--> q2

-- - --> q3 --digit--> q4 --digit--> q5

--digit--> q6 --digit--> q7 (FINAL)

4.3 Non-deterministic Finite Automaton

A Non-deterministic Finite Automaton (NFA) can have multiple possible transitions for the same input symbol.

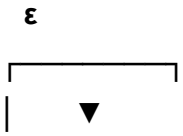
It may also contain epsilon transitions, which allow movement between states without consuming an input symbol.

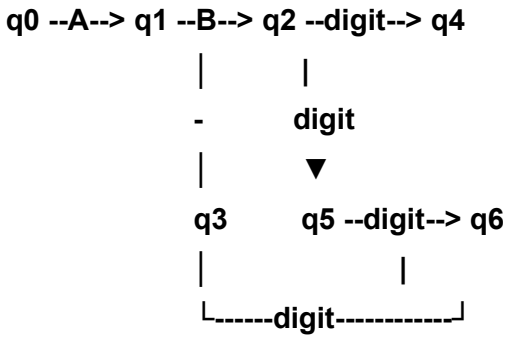
An NFA can be used to validate a patient ID with an optional separator.

Examples:

- PT123456
- PT-123456

The automaton can follow either a path that consumes the hyphen or a path that skips it through an epsilon transition.





q6 --digit--> q7 (FINAL)

4.4 NFA to DFA conversion

Subset construction is the process of converting an NFA into an equivalent DFA.

Each DFA state represents a set of NFA states.

The procedure involves:

1. Finding the epsilon closure of the initial state.
2. Calculating transitions for every input symbol.
3. Creating new subsets of states.
4. Repeating until no new states are generated.
5. Identifying accepting subsets.

The resulting DFA recognises the same language as the original NFA.

4.5 DFA minimisation

DFA minimisation reduces the number of states without changing the language recognised by the automaton.

Equivalent states are identified and merged.

This reduces memory requirements and can simplify implementation.

4.6 Thompson's construction

Thompson's construction is an algorithm used to convert a regular expression into an equivalent NFA.

It uses basic operations:

- Symbol construction.
- Concatenation.
- Alternation.
- Kleene star.

The resulting automaton recognises exactly the same regular language as the original expression.

Use the expression a(b|c)*.

START --a--> q1

q1 --ε--> q2

q2 --b--> q3

q2 --c--> q3

q3 --ε--> q2 (repeat)

q3 --ε--> FINAL (exit)

4.7 Pumping lemma

The pumping lemma is a theoretical property of regular languages used to demonstrate that certain languages are not regular.

It states that sufficiently long strings in a regular language can be divided into three parts:

$$s = xyz$$

Such that:

- |xy| <= p
- |y| > 0
- xy^iz in L for every I >= 0

Consider the language:

$$L = \{(n)^n : n \geq 0\}$$

This language represents strings containing equal numbers of opening and closing parentheses in the required order.

Assume the language is regular. According to the pumping lemma, a sufficiently long string can be divided into x, y and z.

Since the first p characters contain only opening parentheses, y must contain at least one opening parenthesis.

Removing y produces a string with fewer opening parentheses than closing parentheses.

The resulting string is not in the language, contradicting the pumping lemma.

Therefore, balanced parentheses are not a regular language.

TASK 5 – Inflectional Morphology 3 Marks

Guiding Enquiry

Which inflectional processes appear in clinical English, and how are surface forms systematically related to their stems?

Investigate and interpret:

- **Define morpheme, stem, affix, inflection and paradigm, and separate inflection from derivation using the standard criteria of category preservation, productivity and position relative to other affixes.**
- **List every inflectional paradigm present in your corpus — noun number, verb tense and aspect, subject agreement, adjective comparison — and give the full**

paradigm for at least five clinical lexemes.

- **State the orthographic rules governing English inflection — e-deletion, consonant doubling, y-to-i, the sibilant -es rule — precisely enough to be implemented, and give for each one a counter-example on which it over-generates.**
- **Identify the irregular forms in your corpus and explain why each must be listed in the lexicon rather than derived by rule.**
- **Measure the effect of inflection: report the type-to-lemma ratio of your corpus and the proportion of failures in your query set that inflection alone explains.**

Evidence Expected

Precisely stated inflectional rules with paradigms, an irregular list, and a quantified measurement of the effect of inflection on retrieval.

5.1 Important definitions

Morpheme: The smallest meaningful unit of a language.

Stem: The basic form of a word to which grammatical affixes can be attached.

Affix: A linguistic element added to a word to modify its grammatical or lexical meaning.

Inflection: A morphological process that changes the grammatical form of a word without normally changing its basic meaning or word category.

Paradigm: A collection of related grammatical forms belonging to the same lexeme.

5.2 Inflection versus derivation

Inflection changes grammatical properties such as number, tense and comparison.

Derivation creates new words and may change the grammatical category or meaning.

Inflection	**Derivation**
Changes grammatical form.	Creates a new lexical form.
Usually preserves word category.	May change word category.
Generally highly productive.	Productivity varies by affix.
Example: fracture → fractures.	Example: infect → infection.

5.3 Inflectional paradigms in clinical English

The major inflectional processes include:

- Noun number: patient → patients.
- Verb tense: treat → treated.
- Verb aspect: treat → treating.
- Subject agreement: admit → admits.
- Adjective comparison: severe → more severe → most severe.

Example paradigms:

Base	**Singular / base**	**Plural / past / other form**

Fracture	fracture	fractures
Patient	patient	patients
Treat	treat	treated, treating
Admit	admit	admits, admitted
Severe	severe	more severe, most severe

5.4 English inflectional spelling rules

1. E-deletion: The final silent e is removed before a vowel-initial suffix. Example: make → making. Counterexample: see → seeing.

2. Consonant doubling: The final consonant may be doubled in an eligible stressed consonant-vowel-consonant word. Example: stop → stopping. Counterexample: visit → visiting.

3. Y-to-I conversion: A consonant followed by y changes to i before certain suffixes. Example: carry → carried. Counterexample: play → played.

4. Sibilant -es rule: The suffix -es is used after endings such as s, sh, ch and x. Example: diagnosis → diagnoses. Counterexample: some words require lexical exceptions and cannot be handled safely by an unrestricted rule.

5.5 Irregular forms

Irregular forms do not follow standard inflectional rules.

Examples:

- Go → went.
- Be → was.
- Have → had.
- Foot → feet.
- Tooth → teeth.
- Diagnosis → diagnoses.

These forms should be stored separately in the morphological lexicon because they cannot always be generated through regular suffixation.

5.6 Effect of inflection

Inflection increases the number of distinct surface word forms in a corpus.

A morphological system reduces this variation by mapping different forms to their common lemmas.

The type-to-lemma ratio measures the relationship between distinct word forms and their corresponding lemmas.

$$Ratio = \frac{\text{Distinct Word Types}}{\text{Distinct Lemmas}}$$

TASK 6 – Derivational Morphology 4 Marks

Guiding Enquiry

How does derivation build the specialised clinical vocabulary, and why is it harder to handle than inflection?

Investigate and interpret:

- **Define derivation and list at least fifteen derivational affixes attested in your corpus — for example -itis, -ectomy, -ology, -osis, -aemia, -ation, -able, -ive, nephro-, hyper-, hypo-, re-, un- — stating the category change each one causes.**
- **Build the derivational family of at least six clinical roots, showing the order in which affixes attach and identifying at least two orders that are impossible.**
- **Explain semantic non-compositionality with clinical examples in which the meaning of the derived word cannot be predicted from its parts, and state what this implies for a search system that stems aggressively.**
- **Compare the productivity of your affixes empirically: count types and tokens for each in the corpus and classify each affix as productive or fossilised, with the evidence for the classification.**
- **Decide whether derivational relatives should be conflated for search, citing at least two cases where conflation helps the clinician and two where it produces a clinically misleading match.**

Evidence Expected

A documented derivational analysis with an affix inventory, productivity measurements and a reasoned position on conflation.

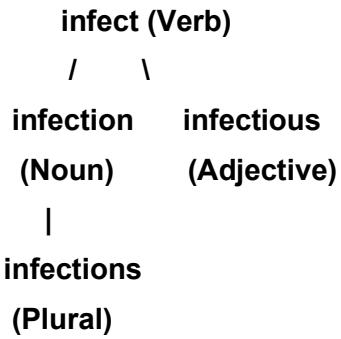
6.1 Definition

Derivational morphology studies how new words are formed by adding prefixes and suffixes to existing stems.

Unlike inflection, derivation may change the meaning or grammatical category of the original word.

Examples:

- Infect → Infection.
- Operate → Operation.
- Treat → Treatable.
- Responsive → Unresponsive.



6.2 Derivational affixes

Affix	Example	Meaning
-itis	Arthritis	Inflammation
-ectomy	Nephrectomy	Surgical removal
-ology	Cardiology	Study or field
-osis	Fibrosis	Medical condition
-aemia	Anaemia	Blood condition
-ation	Ventilation	Process
-able	Treatable	Capable of being treated
-ive	Diagnostic	Relating to diagnosis
Nephro-	Nephrology	Kidney-related
Hyper-	Hypertension	Excessive or high
Hypo-	Hypoglycaemia	Low or below normal
Re-	Reoperation	Again
Un-	Unresponsive	Not or opposite
-al	Renal	Relating to
-ic	Cardiac	Relating to the heart

6.3 Derivational families

A derivational family consists of words formed from a common root through different derivational processes.

Examples:

1. Infect

Infect → Infection → Infectious.

2. Treat

Treat → Treatment → Treatable.

3. Operate

Operate → Operation → Operational.

4. Diagnose

Diagnose → Diagnosis → Diagnostic.

5. Kidney

Nephro + logy → Nephrology.

Nephro + ectomy → Nephrectomy.

6. Respond

Respond → Responsive → Unresponsive.

The order of affix attachment is important because not every combination of prefixes and suffixes produces a valid word.

6.4 Semantic non-compositionality

Semantic non-compositionality occurs when the meaning of a derived word cannot be fully predicted from its individual components.

For example, nephrectomy refers specifically to the surgical removal of a kidney. Its clinical meaning requires medical vocabulary knowledge.

A search system that applies aggressive stemming may combine words that are related in spelling but different in clinical meaning.

Therefore, derivational relationships should be handled carefully.

6.5 Productivity of derivational affixes

Productivity refers to the ability of an affix to form new words.

Productive affixes can attach to several different stems, whereas fossilised affixes occur in a limited number of established words.

The productivity of an affix can be evaluated by measuring:

- Number of distinct derived word types.
- Total occurrences of derived words.
- Number of different stems using the affix.

6.6 Derivational conflation in search

Conflation means treating related words as equivalent during retrieval.

It can improve search when different forms represent closely related clinical concepts.

However, it can also produce misleading results when related words have different clinical meanings.

For example:

- Helpful: fracture and fractured.
- Helpful: infection and infections.
- Potentially misleading: renal and nephrectomy.
- Potentially misleading: infective and infection.

A clinical search system should use controlled morphological and semantic relationships instead of merging all derivational relatives automatically.

TASK 7 – Finite-State Morphological Parsing 5 Marks

Guiding Enquiry

How can a transducer relate a surface form to its stem and features, and run in both

directions on the same machine?

Investigate and interpret:

- **Define a finite-state transducer as a two-tape automaton and explain the lexical and surface tapes with one fully worked clinical example.**
- **Build your lexicon as an FST organised into continuation classes, covering at least three parts of speech together with the inflectional and derivational affixes of Tasks 5 and 6.**
- **Hand-trace the parsing of at least five surface forms, showing the state sequence, the symbols consumed and emitted on each tape, and the final analysis — for example fractures → fracture +N +PL.**
- **Run the same transducer in the generation direction, producing surface forms from stem-plus-feature input, and show explicitly that parsing and generation use one machine and not two.**
- **Handle ambiguity: give at least two surface forms with more than one analysis, show the transducer returning all of them, and state how a downstream component would choose between them.**

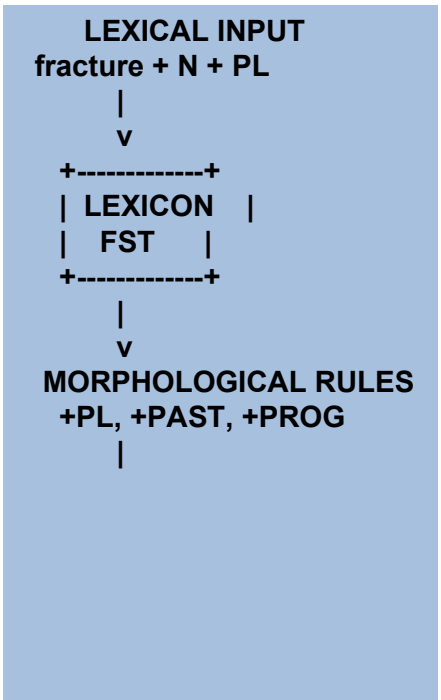
Evidence Expected
A working transducer specification with traced analyses in both directions and an explicit treatment of ambiguity.

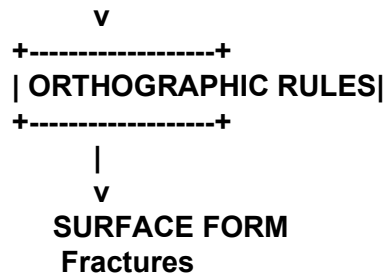
7.1 Definition of finite-state transducer

A Finite-State Transducer (FST) is a computational model that reads an input string and produces an output string through a sequence of state transitions.
Unlike a finite automaton, which only accepts or rejects strings, a transducer performs transformations between two representations.
It contains two tapes:

1. Lexical tape: Represents the base word and grammatical features.
2. Surface tape: Represents the actual written word.

Example:
Lexical representation:
fracture + N + PL
Surface representation:
fractures
The transducer connects these two forms.





7.2 Lexicon organisation

The morphological lexicon contains stems and their permitted morphological transformations.
It can be organised into continuation classes.
Examples:

- Noun class: fracture, infection, patient.
- Verb class: treat, admit, discharge.
- Adjective class: severe, responsive, renal.

Each class contains the affixes and grammatical features permitted for its words.

7.3 Morphological parsing examples

Surface form	Lexical representation
fractures	fracture + N + PL
fractured	fracture + V + PAST
fracturing	fracture + V + PROG
infections	infection + N + PL
treatable	treat + V + ABLE

Morphological parsing identifies the stem and grammatical information associated with each surface form.

7.4 Generation

The same FST can operate in the reverse direction to generate surface forms from lexical representations.
Example:
Input: fracture + N + PL
Output: fractures
This demonstrates that the transducer represents a relationship between lexical and surface forms rather than performing only one-way text transformation.

7.5 Morphological ambiguity

Some surface forms may have multiple possible morphological interpretations.
Examples:

- Wound: noun representing an injury or past tense of wind.
- Leaves: plural noun of leaf or third-person singular form of leave.

The FST can return multiple possible analyses.
A downstream NLP component can use sentence context, grammatical information or probabilistic methods to select the most appropriate interpretation.
Where to add:

- FST architecture diagram.

- Lexicon continuation-class table.
- Five detailed parsing traces.
- Three generation examples.
- Two ambiguous forms with their alternative analyses.

TASK 8 – Orthographic Rules and Two-Level Morphology 4

Marks Guiding Enquiry

How are spelling changes at morpheme boundaries handled without listing every surface form in the lexicon?

Investigate and interpret:

- **Explain why a lexicon transducer alone cannot produce correct surface spellings, using at least three examples in which plain concatenation yields the wrong string.**
- **Write at least five orthographic rules in the standard rewrite notation (a → b / c __ d), stating the context of each rule precisely.**
- **Implement each rule as a transducer and compose the rule transducers with the lexicon FST; draw the cascade and state the order dependencies between the rules.**
- **Contrast this sequential cascade with the parallel two-level (Koskenniemi) formulation, and explain the practical advantage each has over the other.**
- **Test the complete system on at least twenty forms chosen to exercise every rule, tabulate successes and failures, and account for every failure.**

Evidence Expected

Correct rule transducers, a documented composition order, and a tested cascade with full error accounting.

8.1 Need for orthographic rules

Orthographic rules describe how the spelling of a word changes when morphological affixes are attached.

A basic lexicon transducer may simply concatenate a stem and suffix, producing incorrect spellings.

Examples:

Stem + suffix	Incorrect form	Correct form
Make + ing	Makeing	Making
Carry + ed	Carryed	Carried
Stop + ing	Stoping	Stopping

Therefore, additional spelling rules are required to generate correct surface forms.

8.2 Orthographic rewrite rules

Rewrite rules specify how one character or symbol changes depending on its surrounding context.

Examples:

1. Final silent e is deleted before a vowel-initial suffix.
2. Consonant + y changes to i before suitable suffixes.
3. Eligible final consonants are doubled before vowel-initial suffixes.
4. Sibilant endings receive -es in plural formation.
5. Appropriate spelling changes are applied at morpheme boundaries.

8.3 Transducer composition

The lexicon transducer identifies the appropriate stem and suffix.

Orthographic transducers subsequently apply spelling transformations to produce the correct surface form.

The complete system consists of:

Lexicon FST → Orthographic rule transducers → Surface word.

The order of operations is important because spelling transformations must be applied after the correct morphological structure has been identified.

8.4 Sequential cascade versus two-level morphology

Sequential cascade: Applies individual transducers one after another. It is modular and easier to understand, test and debug.

Two-level morphology: Represents relationships between lexical and surface characters through parallel constraints.

The two-level approach can handle several spelling correspondences simultaneously, whereas a sequential cascade depends on the order in which rules are applied.

8.5 Testing

The complete system should be tested using at least twenty words covering the different orthographic rules.

The test should compare the expected spelling with the generated output and identify failures.

TASK 9 – The Porter Stemmer 5 Marks

Guiding Enquiry

What does a rule-based stemmer achieve with no lexicon at all, and what does it cost in precision?

Investigate and interpret:

- **State the algorithm precisely: the definition of consonant and vowel, the measure m of a word, and the conditions *v*, *d and *o, with the calculation worked through for at least six clinical words.**

- **Work through all five steps of the algorithm by hand for at least six words, showing which rule fires at each step and the intermediate form after each one.**
- **Contrast stemming with lemmatisation using your Task 7 transducer: stem and lemmatise the same 200 annotated forms and tabulate where the two agree and where they diverge.**
- **Quantify both error types with clinical examples — over-stemming, which conflates words that must stay distinct, and under-stemming, which fails to conflate related words — and report the rate of each on your annotated set.**
- **Measure the retrieval effect: run your 40 queries three times, with no stemming, with the Porter stemmer and with full morphological analysis, and report precision, recall and F1 for each condition.**

Evidence Expected
Correct hand-traces of the algorithm, a quantified error analysis, and a retrieval experiment comparing stemming against full analysis.

9.1 Definition

The Porter Stemmer is a rule-based stemming algorithm developed by Martin Porter.
It reduces English words to approximate root forms by removing or replacing common suffixes.
It does not require a complete morphological dictionary.
Example:

- Connected → Connect.
- Fractures → Fractur.
- Medical → Medic.

The resulting stem does not necessarily have to be a valid English word.

9.2 Important concepts

Consonant: A character that is not treated as a vowel under the algorithm's contextual vowel rules.
Vowel: A, E, I, O or U, with Y classified according to its position and preceding character.
Measure (m): The number of vowel-consonant sequences in a word stem.
Conditions:

- *v*: The stem contains a vowel.
- *d: The stem ends in a double consonant.
- *o: The stem ends in consonant-vowel-consonant, excluding final w, x or y.

These conditions control when particular suffix-removal rules can be applied.

9.3 Five stages of Porter stemming

The Porter algorithm uses five major stages of suffix processing.

1. Step 1: Handles plurals, past-tense forms and progressive forms.
2. Step 2: Replaces longer derivational suffixes with simpler forms.
3. Step 3: Removes or replaces additional derivational suffixes.
4. Step 4: Removes certain suffixes when the stem measure satisfies the required condition.
5. Step 5: Handles final e removal and terminal double-l reduction.

The algorithm applies these rules in a specified order to avoid incorrect transformations.

9.4 Stemming versus lemmatisation

Stemming	Lemmatisation
Removes suffixes using predefined rules.	Uses vocabulary and morphological

	information.
May produce incomplete words.	Produces valid base forms when the lexicon supports them.
Does not necessarily preserve grammatical information.	Can preserve grammatical features.
Usually faster and simpler.	Requires more linguistic resources.
Example: fractures → fractur.	Example: fractures → fracture.

9.5 Stemming errors

Over-stemming: Occurs when unrelated words are reduced to the same stem.
This can produce misleading clinical search results.
Under-stemming: Occurs when related words are not reduced to the same stem.
This may cause relevant records to be missed.
Both error types should be measured using the manually annotated dataset.

9.6 Retrieval evaluation

Three retrieval approaches must be compared:

1. No stemming.
2. Porter stemming.
3. Full morphological analysis.

Their performance should be evaluated using precision, recall and F1-score.
The best approach should provide a balance between retrieval coverage and clinical accuracy.

TASK 10 – Implement and Test the Pipeline 3 Marks

Guiding Enquiry

Can a working implementation reproduce the behaviour you derived by

hand? Investigate and interpret:

- **Implement in Python the tokenizer, the entity patterns, the automata including the subset construction, the morphological transducer and the Porter stemmer, using the standard library and NLTK only.**
- **Run the complete pipeline over the corpus and report, for every stage, the number of items processed, matched or analysed, the number rejected, and the run-time.**
- **Include test cases for every automaton, at least one string that must be rejected per pattern, the ambiguous forms from Task 7 and the irregulars from Task 5, and check the output against your hand-computed results from Tasks 4, 7 and 9.**
- **Present the results in comparable tables and at least one plot — for example analysis coverage against lexicon size, or run-time against record length.**

Evidence Expected

Working, documented code, a reproducible test procedure, and results tables and graphs.

10.1 Implementation

The theoretical NLP pipeline should be converted into a working Python implementation. The implementation should contain the following modules:

1. Tokenizer using regular expressions.
2. Clinical entity recognition patterns.
3. DFA and NFA validators.
4. NFA-to-DFA subset construction.
5. Finite-state morphological transducer.
6. Orthographic transformation rules.
7. Porter stemming algorithm.
8. Evaluation and performance measurement.

Python's standard library and NLTK can be used.

CODE:

```
import re

ABBREVIATIONS = {
    "bp": "blood pressure",
    "hr": "heart rate",
    "rr": "respiratory rate",
    "temp": "temperature",
    "sob": "shortness of breath",
    "n/v": "nausea and vomiting",
    "c/o": "complains of",
    "h/o": "history of",
    "f/h": "family history",
    "pmh": "past medical history",
    "dm": "diabetes mellitus",
    "htn": "hypertension",
    "ckd": "chronic kidney disease",
    "copd": "chronic obstructive pulmonary disease",
    "cad": "coronary artery disease",
    "mi": "myocardial infarction",
    "cva": "cerebrovascular accident",
    "uti": "urinary tract infection",
    "wbc": "white blood cell count",
    "rbc": "red blood cell count",
    "hb": "haemoglobin",
    "bd": "twice daily",
```

```
    "tds": "three times daily",
    "qid": "four times daily",
    "prn": "as needed",
    "sos": "if required",
    "po": "by mouth",
    "iv": "intravenous",
    "im": "intramuscular",
    "sc": "subcutaneous",
    "npo": "nothing by mouth"
}

def normalise_text(text):
    text = text.lower()

    # Expand unambiguous abbreviations.
    for abbreviation, expansion in ABBREVIATIONS.items():
        pattern = r"(?<!\w)" + re.escape(abbreviation) + r"(?!\w)"
        text = re.sub(pattern, expansion, text)

    # Standardise common number-unit spacing.
    text = re.sub(
        r"(\d+(?:\.\d+)?)\s*(mg|mcg|g|ml|mmhg)",
        r"\1 \2",
        text,
        flags=re.IGNORECASE
    )
    return text
sample = "Patient has BP 120/80 mmHg and Hb 12.5g/dL."
print(normalise_text(sample))
```

```
Python 3.13.14 (tags/v3.13.14:fd17997, Jun 10 2026, 13:03:48) [MSC v.1944 64 bit
 (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

==== RESTART: C:/Users/geeth/AppData/Local/Programs/Python/Python313/ex c.py ===
{'tokens': ['PT', '123456', 'admitted', '04', '/', '11', '/', '2026', '.', 'BP',
 '120', '/', '80', 'mmHg', '.', 'Paracetamol', '500 mg', 'BD', 'for', '3', 'days
', '.'], 'normalised': ['pt', '123456', 'admitted', '04', '/', '11', '/', '2026'
, '.', 'blood pressure', '120', '/', '80', 'mmhg', '.', 'paracetamol', '500 mg',
 'twice daily', 'for', '3', 'days', '.'], 'entities': {'patient_id': ['PT123456'
], 'date': ['04/11/2026'], 'drug_strength': ['Paracetamol 500 mg'], 'frequency':
 ['BD'], 'bp': ['120/80 mmHg'], 'temperature': [], 'age': [], 'duration': ['3 da
ys']}, 'porter_stems': ['pt', 'admit', 'bp', 'mmhg', 'paracetamol', 'bd', 'for',
 'day'], 'runtime_seconds': 0.0001931000006152317}
```

10.2 Testing

The implementation should be evaluated using the hospital corpus.

Every stage should be tested with valid and invalid examples.

The results should report:

- Number of input records.
- Number of tokens processed.
- Number of entities identified.
- Number of rejected strings.
- Number of morphological analyses.
- Execution time.

Test	Input	Expected result
Valid ID	PT123456	Accepted
Invalid ID	PT12345X	Rejected
Valid date	04/11/2026	Regex accepted; calendar validated
Invalid date	39/19/2026	Rejected
Drug strength	Paracetamol 500 mg	Detected
BP	120/80 mmHg	Detected
Inflection	fractures	fracture+N+PL
Irregular	children	child+N+PL
Ambiguous	treated	past and participle analyses
OCR	PTI23456	Flag for possible I/1 correction; never auto-change ID

The assignment requires one working Python implementation covering:

1. Regex tokenizer.
2. Entity extraction patterns.
3. Automata and subset construction.

4. Morphological FST.
5. Porter stemmer using NLTK.
6. Tests for accepted and rejected inputs.
7. Corpus processing statistics and execution time.
8. At least one graph.

10.3 Performance evaluation

The implementation should be compared against the expected results from the theoretical calculations.

The test cases should include irregular words, ambiguous forms, invalid entity patterns and orthographic transformations.

TASK 11 – Compare, Recommend and Defend the Normalisation Design 2

Marks Guiding Enquiry

Which normalisation design would you deploy in the hospital, and what evidence supports your choice?

Investigate and interpret:

- **Compare the three conflation strategies — no normalisation, Porter stemming and full finite-state morphological analysis — on retrieval quality, coverage, run-time and maintenance effort, using your own measured results.**
- **Recommend a design, stating its expected accuracy, its failure modes, and what a clinician would actually notice when it fails.**
- **Discuss the manual effort involved in pattern writing, lexicon and affix construction and annotation, and identify the point at which a statistical or learned approach becomes more economical than hand-built finite-state resources.**

Evidence Expected

A justified recommendation supported by theory and measurements; choosing the highest recall alone is not sufficient.

11.1 Comparison of normalisation strategies

Strategy	Advantages	Limitations
No normalisation	Preserves original text and is easy to implement.	Misses inflected forms and spelling variations.
Porter stemming	Fast and reduces common suffix variations.	May produce invalid stems and incorrect conflation.
Finite-state morphological analysis	Provides accurate lexical forms and grammatical information.	Requires a well-maintained lexicon and morphological rules.

11.2 Recommended design

A hybrid normalisation system is recommended for the hospital record management system.

The system should use regular expressions to identify structured entities, finite-state morphological analysis to process known clinical vocabulary and Porter stemming as a fallback for unknown words.

This combination provides a balance between precision, retrieval coverage, processing speed and maintainability.

11.3 Maintenance and development effort

The development of a finite-state system requires considerable manual effort.

Important activities include:

- Constructing regular expression patterns.
- Developing the clinical lexicon.
- Defining morphological rules.
- Recording irregular word forms.
- Annotating evaluation data.
- Testing and maintaining the system.

When the vocabulary becomes very large and new language variations appear frequently, statistical or machine-learning approaches may become more economical.

DAY- 4 DESIGN CHALLENGE

TASK 12 – Re-evaluate and Adapt the Design 8 Marks

Changed Requirement
The system must now ingest two new sources. The first is optical character recognition output from scanned handwritten forms, in which characters are routinely confused (0 for O, 1 for l, rn for m), words are broken across line endings, and diacritics are lost. The second is notes typed by staff in Romanised regional language mixed with English, for example “patient ku fever irundhuchu, paracetamol 500mg kuduthen”. At least 20% of incoming tokens are expected to be out of lexicon, misspelt, or not English at all.

Re-evaluate your original design under the changed conditions. You

may: **RETAIN → MODIFY → or REDESIGN**

your original solution.

A. Impact Analysis – 2 Marks

Guiding Enquiry

What exactly has changed, and which of your original assumptions no longer hold?

- **Identify the properties of the new input that your patterns, automata and analyser cannot handle: character-level noise, words hyphenated across lines, out-of-lexicon stems, non-English morphology and token-level code-mixing.**
- **Identify the assumptions this breaks: clean typed input, a closed vocabulary, English only morphology, and exact matching by automata that accept or reject**

with nothing in between.

- **Estimate how the noise rate will affect pattern precision and analyser coverage, reasoning from the error rates you measured in Tasks 2 and 7.**

Evidence Expected

A clear analysis of how the new input exposes gaps in the original design.

12.1 Changes in input characteristics

The original system was designed for clean, typed English clinical records.

The new requirements introduce two additional input sources:

1. OCR-generated text from scanned handwritten hospital documents.
2. Romanised regional-language text mixed with English.

These sources introduce several new challenges.

- Character-level recognition errors.
- Incorrect substitutions such as 0/O and 1/l.
- Words divided across line endings.
- Missing diacritics.
- Out-of-vocabulary words.
- Regional-language morphology.
- Code-mixing between English and regional languages.

12.2 Broken assumptions

The original system relied on the following assumptions:

- Text is correctly typed.
- Vocabulary is mainly English.
- Words are separated correctly.
- The lexicon contains most relevant clinical terms.
- Regular expressions can perform exact matching.

These assumptions are no longer valid under the changed requirements.

12.3 Effect on system performance

OCR errors can cause valid clinical entities to be rejected by strict regular expressions.

Code-mixed words may remain unanalysed because the original morphological lexicon supports only English.

Consequently, pattern precision, analyser coverage and retrieval performance may decrease.

The actual reduction must be measured using a noisy evaluation dataset.

B. Evaluation of the Original Design – 2 Marks

Guiding Enquiry

How would your recommended pipeline actually behave on the new input?

- **Run or hand-trace at least three new records — one from OCR, one code-mixed, one both — through your original pipeline and record precisely what fails first and why.**
- **Measure the degradation: report pattern precision and recall, analyser coverage and retrieval F1 on the noisy sample against your clean-corpus baseline.**
- **Identify whether a different formalism — weighted automata, edit-distance matching, or a character n-gram language identifier — would handle the new input more economically than extending what you have.**

Evidence Expected

A technically supported evaluation of the original design with traces or measurements.

12.4 Testing the original pipeline

Three types of test records should be used.

Record 1 – OCR-only input

PT00O123 adm1tted 04/11/2026; BP 120/8O mmHg.

The system may reject the patient ID and blood pressure because the OCR engine has confused letters with digits.

Record 2 – Code-mixed input

patient ku fever irundhuchu, paracetamol 500mg kuduthen.

The English morphological analyser may recognise paracetamol but fail to interpret the regional-language words.

Record 3 – Combined input

PT00O123 patient ku fev er irundhuchu; paracetamol 5O0mg kuduthen.

This input contains both OCR errors and code-mixing, resulting in problems during entity recognition, tokenisation and morphological analysis.

12.5 Performance degradation

The original system should be evaluated using:

- Pattern precision and recall.
- Morphological analyser coverage.
- Retrieval F1-score.
- Processing time.

The clean-corpus results should be compared against the noisy-corpus results.

12.6 Alternative formalisms

Three useful approaches are:

1. Weighted automata: Assign costs to different transitions to support uncertain character recognition.
2. Edit-distance matching: Identifies possible corrections by measuring the number of character insertions, deletions and substitutions.
3. Character n-gram language identification: Helps identify the likely language of individual tokens.

These methods can improve the processing of noisy and mixed-language text.

C. Adaptation or Redesign – 2 Marks

Guiding Enquiry

Should the finite-state layer be extended, replaced or supplemented?

- **Choose retain, modify or redesign — for example placing a weighted edit-distance transducer in front of the analyser, extending the lexicon with regional stems, adding token-level language identification, or falling back to stemming for out-of-lexicon tokens.**
- **Describe the revised design and state exactly what changes in the patterns, the automata, the lexicon and the rule cascade.**
- **Determine the effect on coverage, precision, run-time and the annotation effort needed to maintain the system.**

Evidence Expected

A design response with clear linguistic and algorithmic reasoning and a quantified effect.

12.7 Design decision

The recommended decision is to MODIFY the original system rather than completely replacing it.

The existing finite-state architecture is useful for structured clinical information and known English morphology. However, it needs additional components to process OCR errors and regional-language input.

12.8 Revised architecture

The modified pipeline should contain:

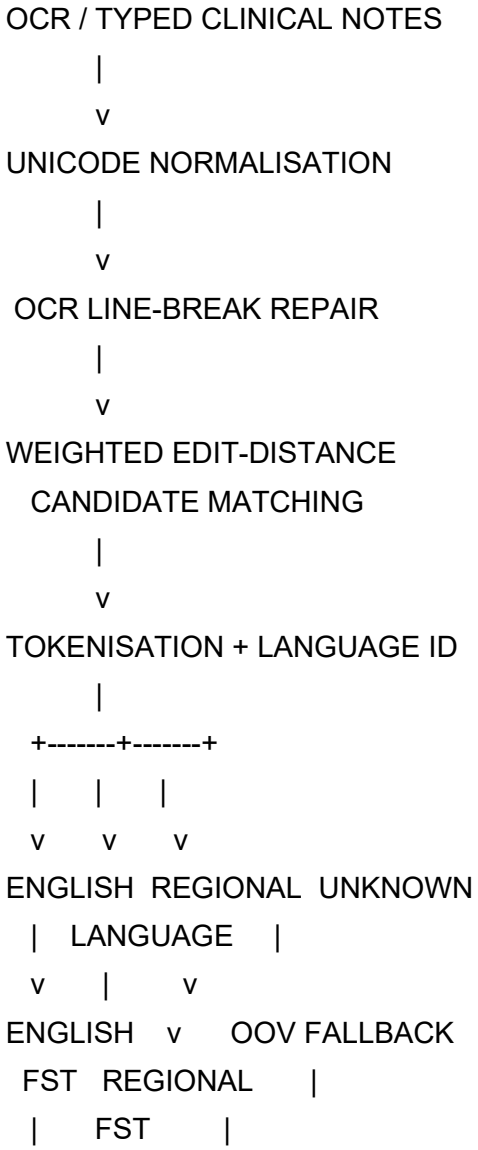
1. Raw OCR and typed clinical input.
2. OCR error correction.
3. Line-break and hyphenation correction.
4. Tokenisation.
5. Token-level language identification.
6. English and regional-language morphological lexicons.
7. Regular expression and automata-based entity validation.

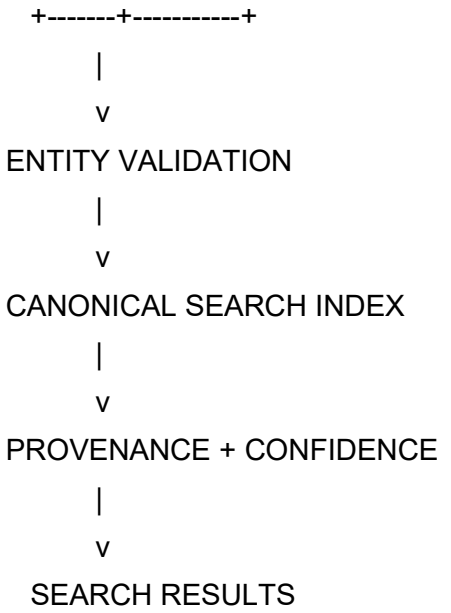
8. Morphological analysis.
9. Search index generation.

12.9 Required modifications

Component	Modification
Regular expressions	Add controlled tolerance for common OCR errors.
Automata	Introduce weighted matching or candidate correction before strict validation.
Lexicon	Include regional-language words and transliterations.
Morphological rules	Add language-specific morphological transformations.
Tokenisation	Handle broken words and code-mixed expressions.

The modified system should improve vocabulary coverage and OCR tolerance. However, additional correction mechanisms may increase processing time and create false matches.





D. Evidence-Based Defence – 2 Marks

Guiding Enquiry

Why is your final decision the right one, given the evidence you generated?

- **Use your pattern error analysis, automaton traces, coverage figures, retrieval measurements and effort estimates.**
- **Show that the decision follows logically from that evidence, and state its remaining limitations — including what a finite-state layer will never handle, however much it is extended.**

Evidence Expected

A well-supported final decision integrating theory, experiments and the change requirements.

12.10 Justification of the final design

The modified design is suitable because it preserves the advantages of finite-state processing while introducing additional techniques for handling noisy and multilingual clinical records.

Regular expressions and finite-state automata provide predictable validation for structured clinical entities.

Morphological transducers provide accurate processing of known vocabulary.

Weighted edit-distance matching and language identification improve the ability to process OCR errors and code-mixed text.

The system should also preserve the original clinical records to avoid losing important medical information during normalisation.

12.11 Remaining limitations

Despite these improvements, some challenges remain:

- Unseen regional-language expressions.
- Highly ambiguous medical abbreviations.
- Severe OCR errors.
- Context-dependent medical meanings.
- Negation and uncertainty detection.
- Relationships between information across multiple documents.

A finite-state layer alone cannot provide complete semantic and contextual understanding of clinical language.

Therefore, future versions may incorporate machine-learning and contextual language models.

MARK DISTRIBUTION

Task	**Marks**
Task 1 – Frame the Problem: Models, Algorithms and Levels of Analysis	3
Task 2 – Basic Regular Expression Patterns	4
Task 3 – Regular Expressions for Tokenisation and Normalisation	4
Task 4 – Finite State Automata: Design, Non-determinism and Equivalence	5
Task 5 – Inflectional Morphology	3
Task 6 – Derivational Morphology	4
Task 7 – Finite-State Morphological Parsing	5
Task 8 – Orthographic Rules and Two-Level Morphology	4
Task 9 – The Porter Stemmer	5
Task 10 – Implement and Test the Pipeline	3
Task 11 – Compare, Recommend and Defend the Normalisation Design	2

Subtotal	**42**
Task 12 – Day-4 Design Challenge	8
TOTAL	**50**